# Mixture of Experts in LLMs

Rapport de stage

Mai - Juillet 2025

Pierre-Louis Filoche

Supervisé par Loris Marchal et Pablo Piantanida

**Résumé**

Les architectures (MoE) permettent d'augmenter la capacité des modèles à coût de calcul quasi-constant grâce à une *activation parcimonieuse* d'experts, pilotée par un routeur. Ce stage étudie la possibilité de *prédire*, à partir du contexte textuel immédiat, *les experts activés* par un LLM MoE au fil des couches, afin (i) d'anticiper des chargements d'experts (préchargement) et (ii) de mieux comprendre la dynamique de routage. Nous proposons plusieurs prédicteurs : statistiques déterministes (profondeur $1-5$), MLP léger puis un LSTM opérant sur des embeddings de tokens *réduits* par PCA (ou réduction apprise). Nous analysons les compromis précision/taille, l'impact de la profondeur de contexte, et le rôle de la réduction dimensionnelle des embeddings.

# Table des matières

**1 Introduction** **3**

**2 Environnement de travail** **5**
2.1 Présentation du laboratoire - International Laboratory on Learning Systems (ILLS) . . . . . 5
2.2 Ressources . . . . . 5
2.2.1 Infrastructure - Grid'5000 . . . . . 5
2.2.2 Modèle étudié : Mixtral 8×7B . . . . . 6

**3 Contexte et Motivation** **7**

**4 Méthodologie** **9**
4.1 Introduction . . . . . 9
4.2 Définition des métriques de précision . . . . . 9
4.3 *Gating Network* . . . . . 11
4.4 Prédicteurs déterministes par statistiques . . . . . 12
4.5 Heatmaps d'utilisation des experts . . . . . 14

**5 Prédicteurs déterministes** **18**
5.1 Prédicteur statistique simple (profondeur 1) . . . . . 18
5.2 Extension à une profondeur supérieure . . . . . 19
5.3 Prédicteur contextuel avec décalage . . . . . 22
5.4 Précision à l'échelle du vocabulaire . . . . . 23

**6 Prédicteurs automatiques** **27**
6.1 Préparation des données . . . . . 27
6.2 Modèle MLP . . . . . 27
6.3 Modèle LSTM . . . . . 28

**7 Conclusion et Perspectives** **30**

**8 Bibliographie** **33**

# 1 Introduction

Depuis plus de trente ans, *scaler* la taille des réseaux de neurones a été l'une des stratégies les plus sûres pour améliorer leurs performances : plus de données, plus de paramètres, de meilleurs résultats. Cependant à chaque augmentation de taille des réseaux, la taille des calculs et de la mémoire nécessaire. Dans le contexte des grands modèles de langage (LLM), cette tension devient critique : on souhaite des milliards, voire des centaines de milliards de paramètres, sans pour autant multiplier d'autant le coût d'inférence. C'est précisément pour résoudre ce dilemme qu'a été introduite l'architecture *Mixture of Experts* (MoE).

Le principe est conceptuellement simple : **ne pas activer tous les paramètres du réseau pour chaque token**. Le cœur d'un bloc MoE est un ensemble d'`experts`, des sous-réseaux feed-forward indépendants, et un réseau de routage. La `gating-function` (réseau de routage) choisit un sous-ensemble réduit d'experts à exécuter pour chaque couche. Cette activation *sparse* permet d'augmenter la capacité totale (nombre de paramètres) sans augmenter proportionnellement les flottantes-opérations, réalisant ainsi le slogan « *capacity without compute* ». Historiquement, les premières versions d'experts remontent aux années 1990, mais c'est l'explosion des besoins de capacité dans les LLM qui a ravivé l'intérêt pour les MoE.

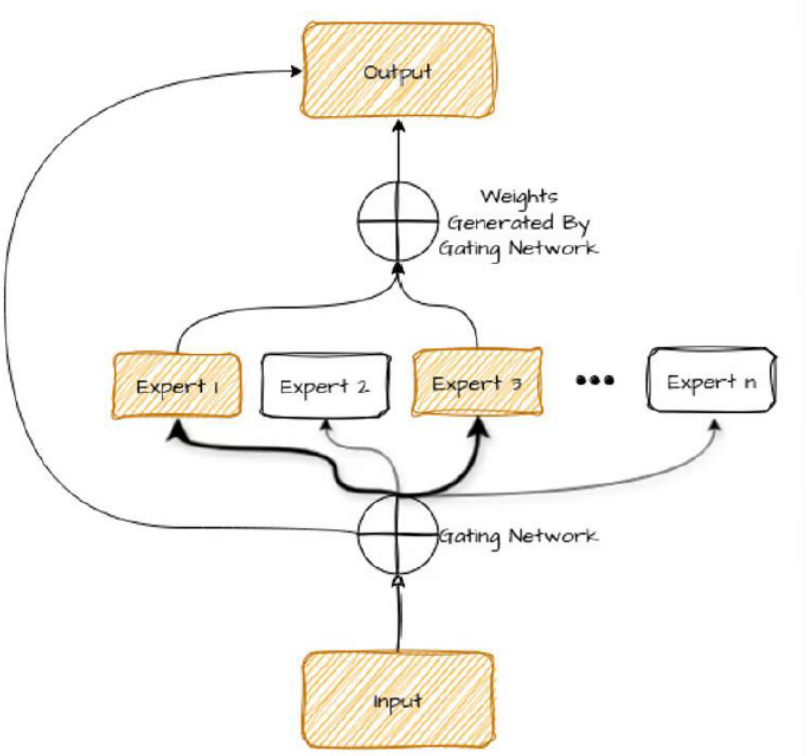



Figure 1 : Architecture d'un bloc Mixture of Experts (MoE)

Outre leur efficacité, les MoE présentent deux atouts essentiels :

1. **Spécialisation émergente.** Sous la pression de l'apprentissage, les experts apprennent souvent des compétences différentes (syntaxe, ponctuation, entités rares,

etc.). Comprendre et contrôler ce phénomène est aujourd'hui un sujet de recherche actif.

2. **Flexibilité systèmes.** Comme seuls quelques experts sont chargés en mémoire GPU à chaque pas, on peut déporter les autres sur CPU / SSD ou adopter des stratégies de quantification agressives, ouvrant la voie à des déploiements sur matériel contraint.

Cependant, cette parcimonie pose aussi de nouveaux défis : équilibrage de charge entre experts, stabilité de l'entraînement, sécurité (fuites d'information par le routage) ou encore intégrité face aux attaques par débordement de capacité. Les chapitres qui suivent analyseront ces questions et détailleront notre propre exploration des MoE, menée sur l'infrastructure Grid'5000 avec le modèle `Mixtral-8×7B`.

Ainsi, les MoE s'imposent aujourd'hui comme une réponse pragmatique à la course à l'échelle : ils réconcilient l'appétit paramétrique des LLM avec les contraintes très réelles de calcul et de mémoire, tout en ouvrant un champ fertile pour la recherche sur la modularité et l'explicabilité des réseaux de neurones.

# 2 Environnement de travail

## 2.1 Présentation du laboratoire - International Laboratory on Learning Systems (ILLS)

L'*International Laboratory on Learning Systems* (ILLS) est un International Research Laboratory (IRL) créé en 2022. Il réunit CentraleSupélec, l'Université Paris-Saclay, le CNRS, McGill University, l'École de technologie supérieure (ÉTS) et Mila, l'Institut québécois d'intelligence artificielle.

L'ILLS a pour vocation de développer des outils mathématiques afin d'améliorer les algorithmes d'apprentissage automatique et de sécuriser leur usage. Les méthodes étudiées s'appliquent notamment au traitement automatique des langues et de la parole, à la vision par ordinateur, ainsi qu'au traitement du signal et de l'information.

Les activités du laboratoire s'articulent autour de cinq axes principaux : les aspects fondamentaux de l'intelligence artificielle pour en sécuriser l'usage, l'apprentissage séquentiel en temps réel, les systèmes autonomes robustes, le traitement du langage naturel et de la parole, et les applications à la vision par ordinateur, aux signaux et au traitement de l'information.

En tant que pôle international de compétences pluridisciplinaires, l'IRL ILLS vise à structurer et intensifier les collaborations franco–canadiennes aussi bien en recherche fondamentale en intelligence artificielle que dans ses applications aux systèmes d'apprentissage.

## 2.2 Ressources

### 2.2.1 Infrastructure - Grid'5000

Les expériences ont été conduites sur **Grid'5000**, une grande plateforme scientifique française multi-sites dédiée à l'expérimentation reproductible en calcul intensif, réseaux et systèmes distribués. Elle offre un accès partagé à un parc matériel hétérogène (CPU/GPU) et à des fonctionnalités bas niveau (réservation exclusive, déploiement d'images, réseau).

**Réservation des nœuds (OAR).** **OAR** est le gestionnaire de ressources et planificateur (*batch scheduler*) utilisé sur Grid'5000 : il alloue les nœuds demandés pour une durée donnée selon la disponibilité et les files d'attente. Les réservations se font via `oarsub` (jobs batch ou interactifs), en spécifiant ressources (CPU/GPU/mémoire), durée et contraintes éventuelles.

**Matériel utilisé.**

- **2× NVIDIA A40** — 40 Go de VRAM par GPU (sessions multi-GPU).
- **1× NVIDIA H100** — 90 Go de VRAM (sessions mono-GPU hautes performances).

### 2.2.2 Modèle étudié : Mixtral 8×7B

Mixtral 8×7B est un modèle de langage *Mixture-of-Experts* parcimonieux d'environ 47 milliards de paramètres au total, dont près de 13 milliards seulement sont effectivement activés à l'inférence grâce à un routage top-2 parmi 8 experts à chaque couche, sur une profondeur de 32 couches Transformer. Ce choix offre une grande capacité effective sans coût de calcul proportionnel, ce qui en fait un cadre pertinent pour étudier la prédiction du routage et le préchargement d'experts. Nous utilisons l'implémentation `huggingface/transformers`, avec une empreinte mémoire d'environ 45 Go en demi-précision, compatible avec notre parc (2× A40 40 Go ou 1× H100 90 Go). Le modèle a été retenu pour sa disponibilité, son adoption large et l'accès aisé aux *router logits* et états intermédiaires requis pour nos analyses.

# 3 Contexte et Motivation

Les modèles de langage ne cessent de croître en taille, ce qui augmente le coût d'inférence. Les MoE apportent une réponse partielle à ce problème : à chaque passage, le routeur n'active qu'un sous-ensemble d'experts par *token* (*top-k* gating), de sorte qu'on n'utilise effectivement qu'une fraction $n/N$ des poids du modèle, où $n$ est le nombre d'experts activés parmi $N$ disponibles .On obtient ainsi une *parcimonie de calcul* (moins de FLOPs par token), mais pas une *parcimonie mémoire* : l'ensemble des experts doit rester chargé ou rapidement chargeable. C'est précisément ce décalage entre parcimonie computationnelle et contrainte mémoire qui motive notre approche : prédire à l'avance les experts qui seront sollicités afin prévoir le préchargement des différents experts et de lisser l'inférence.

**Motivation : préchargement des experts.** La méthode MoE apporte une parcimonie *de calcul* mais pas *de mémoire* : à chaque couche, seuls $n$ experts sur $N$ sont activés, mais l'ensemble des poids des $N$ experts doit rester accessible. Deux situations extrêmes se présentent. Si la VRAM est suffisante pour conserver tous les experts, l'inférence n'est pas ralentie par des transferts, mais le coût mémoire devient prohibitif et on ne tire plus pleinement parti de la parcimonie du MoE. À l'inverse, si la VRAM est limitée et ne peut contenir que quelques experts par GPU, les poids manquants doivent être chargés à la demande depuis la RAM ou un autre GPU via PCIe/NVLink : ces E/S sont bien plus lentes que le calcul en VRAM, introduisent des chargements bloquants, et dominent rapidement la latence. C'est précisément pour combler ce décalage que nous proposons de *prédire* à l'avance les experts qui seront activés et de les *précharger* (idéalement sur des GPU distincts) afin de supprimer les E/S et d'augmenter le parallélisme.

**De la prédiction au préchargement.** En effet savoir *à l'avance* quels experts seront sollicités pendant le *forward* permet de lancer leur transfert en VRAM de façon asynchrone et d'en *masquer* le coût sous le calcul en cours : le chargement n'est alors plus bloquant, pourvu qu'il démarre suffisamment tôt. L'objectif opérationnel devient donc d'obtenir une prédiction d'activation d'experts assez en amont pour respecter cette fenêtre de préchargement, tout en conservant une précision suffisante pour éviter des transferts inutiles.

Nous distinguons deux types de prédiction. La *prédiction spatiale* vise, pour un *token* donné $t$, à estimer l'ensemble d'experts utilisés à la couche $\ell$ du même *forward* (routage *within a token, across layers*). Elle peut exploiter des signaux très récents, tels que l'embedding du *token*, les états cachés des couches précédentes ou même les *router logits* déjà disponibles aux couches $< \ell$, ce qui tend à la rendre plus précise, mais elle n'offre qu'un *délai* réduit pour précharger. À l'inverse, la *prédiction temporelle* estime, pour une couche $\ell$, les experts qui seront activés au *token* $t{+}1$ (ou $t{+}\Delta$) à partir d'informations plus éloignées dans le temps. Elle fournit une anticipation plus précoce, donc une meilleure marge pour les E/S, au prix d'une incertitude plus élevée.

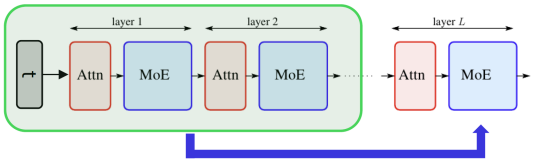



FIGURE 1 – Prédiction Spatiale

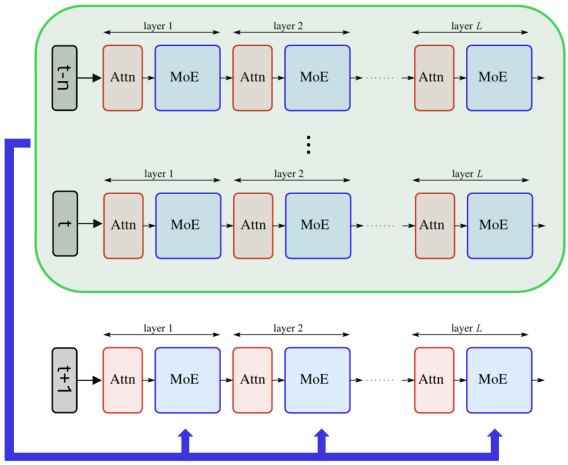



FIGURE 2 – Prédiction temporelle

Ces deux approches répondent à la même problématique sous des angles différents et s'assemblent naturellement dans un schéma *cascadé* : une prédiction temporelle grossière déclenche un préchargement initial, puis la prédiction spatiale, plus tardive mais plus fiable, affine ou annule ce préchargement en fonction d'un seuil de confiance. Par ailleurs, la prédiction temporelle éclaire une décision cruciale de *caching* : à la couche $\ell$ du *token* $t$, savoir si l'on doit conserver en mémoire les experts pour la même couche au *token* $t{+}1$ permet d'éviter une E/S de va-et-vient lorsque les experts sont identiques (ou partiellement communs). En pratique, on pourrait choisir ainsi une fenêtre de préfetch par couche, un budget mémoire et des seuils de confiance pour arbitrer entre transferts anticipés, maintien en cache et éviction, de façon à réduire l'enchaînement des E/S et à réduire la latence d'inférence.

# 4 Méthodologie

## 4.1 Introduction

Après avoir présenté les intérêts et les limites de l'approche Mixture-of-Experts, défini nos objectifs en termes de prédiction et identifié les deux grandes familles de stratégies prédictives (spatiale et temporelle), il reste à déterminer les outils méthodologiques permettant de mettre en œuvre cette idée. Notre objectif opérationnel est de prédire les *routing paths*, c'est-à-dire les experts effectivement sélectionnés par le modèle au cours de l'inférence.

Le modèle étudié est Mixtral 8×7B, constitué de 32 couches, chacune comportant 8 experts dont seulement 2 sont activés par token à chaque passage. Le mécanisme de sélection repose sur un *gating network* (voir figure 1), qui prend en entrée le *hidden vector* d'une couche et produit en sortie huit *logits*. Ces valeurs reflètent la pertinence de chaque expert et permettent de choisir les deux les plus cohérents pour transformer la représentation courante. Prédire les experts utilisés revient donc à estimer la sortie de ce *gating network* avant son exécution. La question centrale devient alors : *quelles informations exploiter pour effectuer cette prédiction et, surtout, lesquelles apportent un véritable signal utile ?*

Un autre enjeu majeur concerne la taille du prédicteur lui-même. En théorie, on pourrait envisager un modèle de grande capacité, exploitant tout l'historique des trajectoires de routage pour obtenir une précision élevée. Cependant, un tel prédicteur serait aussi coûteux en mémoire et en calcul que le modèle génératif qu'il accompagne, ce qui annulerait l'intérêt pratique de la démarche. Notre objectif est au contraire de concevoir un prédicteur léger, capable de s'intégrer à l'architecture existante sans surcharge significative en ressources. L'efficacité de la solution repose donc sur un équilibre délicat : le prédicteur doit être suffisamment expressif pour fournir des estimations fiables, tout en restant assez compact pour avoir un impact réel et mesurable dans un scénario d'inférence à grande échelle.

## 4.2 Définition des métriques de précision

Pour évaluer la qualité de nos prédicteurs d'experts, nous avons défini des métriques adaptées à la nature particulière du routage dans les modèles *Mixture of Experts* (MoE). En effet, à chaque couche du modèle, deux experts parmi huit sont sélectionnés. Une prédiction n'est donc pas une classe unique, mais un couple d'experts.

Afin de tenir compte de cette spécificité, nous avons retenu deux métriques principales : le **top-1** et le **top-2**.

**Top-1.** La prédiction est considérée correcte si l'expert auquel notre prédiction attribue la probabilité maximale appartient effectivement à la paire d'experts réellement activée. Formellement, si $E_\ell = \{e_\ell^{(1)}, e_\ell^{(2)}\}$ désigne la paire d'experts activée à la couche $\ell$, et $\hat{e}_\ell^{(1)}$ l'expert prédit comme le plus probable, alors :

$$\text{Top-1} = \frac{1}{N} \sum_{\ell=1}^{N} \mathbf{1}\left[\hat{e}_\ell^{(1)} \in E_\ell\right],$$

où $N$ est le nombre total de prédictions évaluées et $\mathbf{1}[\,\cdot\,]$ l'indicatrice.

**Top-2.** Ici, nous considérons explicitement les deux experts prédits, notés $\hat{e}_\ell^{(1)}$ et $\hat{e}_\ell^{(2)}$. Le score attribué est 0 si aucun des deux n'appartient à la paire réelle, 1 si un seul est correct et 2 si les deux sont corrects. Le résultat final est normalisé par le nombre de tests :

$$\text{Top-2} = \frac{1}{2N} \sum_{\ell=1}^{N} \left( \mathbf{1}\left[\hat{e}_\ell^{(1)} \in E_\ell\right] + \mathbf{1}\left[\hat{e}_\ell^{(2)} \in E_\ell\right] \right).$$

**Interprétation.** Le *top-1* fournit une mesure stricte mais souvent plus clémente : il suffit de deviner un expert pertinent pour réussir. Le *top-2*, en revanche, est plus exigeant puisqu'il évalue la correspondance sur l'ensemble de la paire. Dans la pratique, cela explique que la précision mesurée en top-2 soit inférieure à celle obtenue en top-1, en particulier lorsque le premier expert est bien prédit mais que le second est plus instable.

Ainsi, l'évaluation croisée par ces deux métriques permet de distinguer la capacité du modèle à identifier les experts dominants (top-1) et sa faculté à reconstituer fidèlement les couples d'experts utilisés (top-2).

### Précision attendue pour une stratégie aléatoire

Afin de disposer d'un point de comparaison, nous avons calculé le taux de réussite attendu pour une stratégie purement aléatoire pour la métrique *top-2*. Dans ce cas, le modèle choisit au hasard deux experts parmi les 28 couples possibles. L'arbre de probabilité associé est représenté figure 3 où $X$ représente le nombre d'experts correctement prédit.

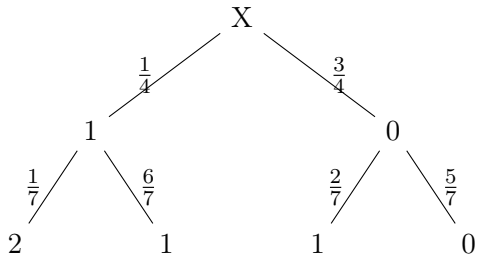



FIGURE 3 – Arbre de probabilité pour une stratégie aléatoire.

L'espérance de réussite peut se calculer ainsi :

$$E(X) = 2 \times \frac{1}{28} + 1 \times \left( \frac{6}{28} + \frac{6}{28} \right) + 0 \times \frac{15}{28} = 0.5,$$

ce qui correspond à un *hit rate* moyen de 0.25.

Autrement dit, un prédicteur aléatoire atteint en moyenne une précision de 25 en *top-2*. Pour avoir un intérêt pratique, nos modèles prédictifs doivent donc dépasser ce seuil de base, qui constitue une référence minimale.

## 4.3 *Gating Network*

Le *gating network* est entraîné conjointement avec le modèle principal au cours de la phase d'apprentissage supervisé. À l'inférence, il devient déterministe : pour un état caché donnés, le choix des experts est fixe. Une approche naïve mais naturelle consisterait donc à réutiliser ce réseau de *gating*, mais en le nourrissant d'informations plus anciennes. Comme le mécanisme est déterministe et répliqué dans chaque couche, il peut être extrait et appliqué à volonté. Par exemple, pour un token $t$, on peut envisager d'utiliser la fonction de *gating* de la couche $\ell$ mais appliquée au *hidden vector* produit par une couche antérieure $\ell - k$. Le paramètre $k \in \mathbb{N}$ correspond alors à la profondeur de la prédiction spatiale : plus $k$ est grand, plus la prédiction est anticipée, mais potentiellement moins précise.

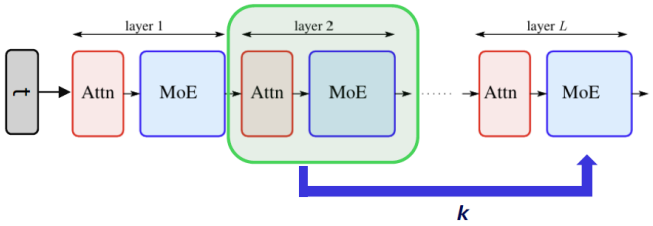



FIGURE 4 – Prédiction via le *Gating Network*

Dans ce cadre, on se place à la couche $k$ et l'on applique la fonction de *gating* de la couche $k+n$ au *hidden vector* issu de $k$, afin d'anticiper les experts qui seront sélectionnés $n$ couches plus loin. L'évaluation se fait selon deux critères. Pour le *top-1*, on vérifie si l'expert prédit avec la plus forte probabilité appartient bien à la paire d'experts réellement activée. Pour le *top-2*, on compte un succès si au moins l'un des deux experts prédits est effectivement utilisé, et deux succès si la paire prédite correspond exactement à la paire activée.

**Extraction des fonctions de gating dans Mixtral**

Pour accéder aux fonctions de *gating* de Mixtral, nous avons modifié le code source de l'implémentation Hugging Face. Chaque bloc MoE (`MixtralSparseMoeBlock`) contient une fonction de routage qui calcule, pour un état caché donné, un vecteur de 8 logits correspondant aux scores des experts. En instrumentant cette fonction et en interceptant sa sortie, nous pouvons enregistrer les logits bruts avant la sélection des deux experts les plus probables. Cette instrumentation permet ainsi d'extraire les distributions complètes de routage, et pas seulement les experts finalement choisis, ce qui constitue une information précieuse pour analyser la dynamique du modèle et entraîner des prédicteurs. De plus on utilise les fonctions intégrées de HuggingFace pour récupérer les *routeur_logits*.

Les résultats montrent une bonne précision : environ 96 % en top-1 et 90 % en top-2 pour $n = 1$, et autour de 85 % pour le *top-2 exact*. Le principal inconvénient de cette

méthode est qu'elle demande de relier explicitement les couches entre elles, mais ce surcoût reste négligeable par rapport au calcul des experts eux-mêmes. Son avantage majeur est qu'elle permet de déclencher un préchargement juste une couche avant l'utilisation effective des experts : c'est donc ce scénario $n = 1$ qui est le plus pertinent en pratique, puisqu'il maximise le rapport entre précision et utilité opérationnelle.

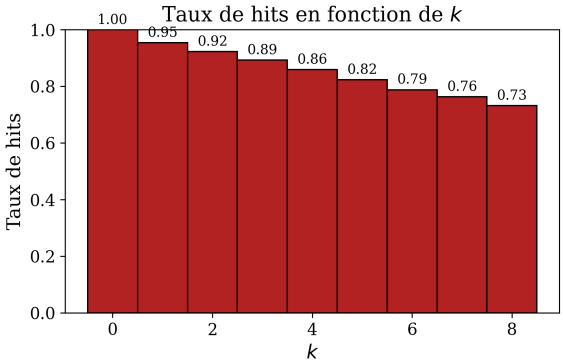



FIGURE 5 – Prédiction top-1

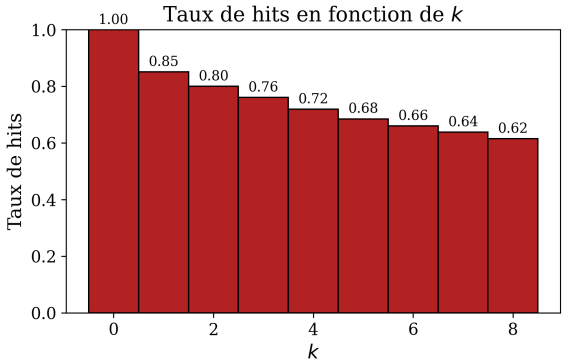



FIGURE 6 – Prédiction top-2

**Conclusion.** L'utilisation directe du *gating network* pour anticiper les experts activés met en évidence un signal fort et exploitable. Les résultats obtenus, notamment pour une prédiction top-1, atteignent des taux de précision élevés tout en restant peu coûteux à mettre en œuvre. Cette approche simple démontre donc la faisabilité du préchargement conditionnel des experts et valide l'idée que le routage n'est pas aléatoire mais fortement structuré dans le modèle. Néanmoins, ses limites apparaissent dès que l'on cherche à anticiper plus en amont ($k > 1$), avec une chute progressive des performances. La suite de notre travail s'oriente ainsi vers des prédicteurs plus flexibles, capables d'intégrer davantage de contexte tout en conservant un coût computationnel réduit, afin de prolonger l'horizon de prédiction et de rendre le préchargement plus efficace.

## 4.4 Prédicteurs déterministes par statistiques

Après avoir exploité les *gating networks* pour prédire les trajectoires d'experts, nous avons cherché à concevoir des prédicteurs plus en amont, c'est-à-dire capables de fournir une estimation avant que les signaux issus des couches profondes ne soient disponibles. Cette recherche implique de trouver une nouvelle source d'information : suffisamment riche pour contenir un signal pertinent, mais suffisamment compacte pour ne pas alourdir la mémoire ou le calcul. La difficulté est que l'on ne sait pas précisément selon quels critères les experts sont choisis par le *gating network*.

Un travail antérieur [8] a montré que la provenance grammaticale des tokens pouvait partiellement expliquer la répartition des trajectoires d'experts. Par exemple, certains experts sont plus souvent associés à la ponctuation, d'autres aux noms propres ou aux

structures syntaxiques particulières. Ces observations suggèrent que les informations lexicales et grammaticales des tokens contiennent déjà un signal exploitable pour prédire le routage.

Partant de ce constat, nous avons choisi de n'utiliser qu'une information minimale : les *token_id*. Dans les modèles de langage, un *token_id* est un entier unique associé à un segment de texte (mot, sous-mot ou caractère) dans le vocabulaire. Chaque token d'une séquence textuelle est ainsi converti en identifiant numérique avant d'être projeté dans l'espace des embeddings. Autrement dit, la séquence de *token_id* constitue la représentation discrète la plus compacte du texte en entrée.

Pour tester l'hypothèse qu'il existe un signal prédictif dans les séquences discrètes de *token_id*, nous avons étudié les matrices de co-occurrence entre tokens et experts. L'idée est la suivante : pour un *token_id* donné, on observe sur l'ensemble du jeu de données quels experts sont activés à chaque occurrence, et on enregistre les couples d'experts effectivement sélectionnés par le routeur. Ce comptage systématique permet ensuite de construire une matrice de co-occurrence qui reflète les régularités statistiques du routage.

**Extraction pratique des chemins de routage**

Concrètement, nous utilisons un script dédié (`Generate_output.py`) qui, pour chaque token du corpus, enregistre l'identifiant du token, les logits produits par le réseau de routage (taille $32 \times 8$), ainsi que les états cachés (taille $33 \times 4096$).

`[token_id, router_logits_{32×8}, hidden_states_{33×4096}]`.

Ces tensors sont stockés au format `.pt` et post-traités par `Stat_experts.py` afin de récupérer, pour chaque couche $\ell$, la paire $(e_\ell^{(1)}, e_\ell^{(2)})$ effectivement sélectionnée.
Le choix d'enregistrer 33 états cachés (et non 32) provient du fait que le premier état correspond à la sortie de l'embedding, avant l'application de la première couche. On dispose donc à la fois de l'entrée initiale et des sorties successives des 32 couches du modèle, ce qui permet de conserver une vision complète de la trajectoire d'un token.

Une fois ces chemins de routage extraits, nous construisons une matrice de co-occurrence où chaque ligne correspond à un couple d'experts à la couche $L$, et chaque colonne à un couple d'experts à la couche $L+1$. La question centrale est alors celle de la normalisation de cette matrice.

**Normalisation par ligne.** Lorsque la matrice de co-occurrence (de taille $28 \times 28$) est normalisée par ligne, chaque élément correspond à une probabilité conditionnelle :

$$P\big(\text{couple}_j \text{ en couche } L+1 \;\big|\; \text{couple}_i \text{ en couche } L\big).$$

Autrement dit, on mesure la probabilité que le couple $j$ soit activé à la couche suivante, sachant que le couple $i$ l'a été à la couche précédente. Cette approche permet d'analyser les transitions inter-couches et de mettre en évidence des règles locales de passage. Toutefois, elle peut être trompeuse lorsqu'un couple $i$ est très rare : si, dans les quelques cas où il

apparaît, il mène toujours au même couple $j$, alors $P(j|i) \approx 1$ bien que le chemin $i \to j$ soit extrêmement marginal en fréquence absolue.

**Normalisation globale.** On peut au contraire normaliser la matrice sur l'ensemble de ses éléments, ce qui donne une probabilité jointe :

$$P\big(\text{couple}_i \text{ en couche } L \ \wedge \ \text{couple}_j \text{ en couche } L+1\big).$$

Chaque case $(i, j)$ reflète alors directement la fréquence absolue du chemin $i \to j$ dans les données. Cette approche met en avant les transitions réellement fréquentes, pondère les liens conditionnels par leur occurrence effective et permet de détecter les chemins dominants dans la dynamique globale du routage. Elle évite ainsi de surinterpréter des chemins rares, mais elle ne permet pas d'identifier les régularités locales : un couple $j$ peut apparaître très souvent simplement parce qu'il est populaire globalement, indépendamment de $i$.

**Visualisation.** Pour obtenir une vue complète, nous traçons la matrice de co-occurrence avec les deux types de normalisation. La première éclaire les régularités conditionnelles entre couches, tandis que la seconde révèle les chemins statistiquement dominants dans la pratique. La combinaison de ces deux perspectives permet de mieux comprendre la structure interne du routage et d'évaluer le potentiel d'un prédicteur basé uniquement sur les *token_id*.

## 4.5 Heatmaps d'utilisation des experts

Afin de mieux comprendre les régularités statistiques entre tokens et experts, nous avons représenté graphiquement l'utilisation des experts en fonction des couches à partir des trajectoires extraites. Dans un premier temps, nous considérons l'ensemble des tokens d'un petit jeu de données (10 *prompts*, soit 448 tokens). La figure 7 montre que, même sur un nombre restreint d'entrées, l'utilisation des experts apparaît relativement uniforme à travers les couches. Ce résultat est cohérent avec la volonté, lors de l'entraînement, d'équilibrer l'appel aux différents experts en ajoutant du bruit au routeur (*router noise*), afin de solliciter l'ensemble des poids disponibles.

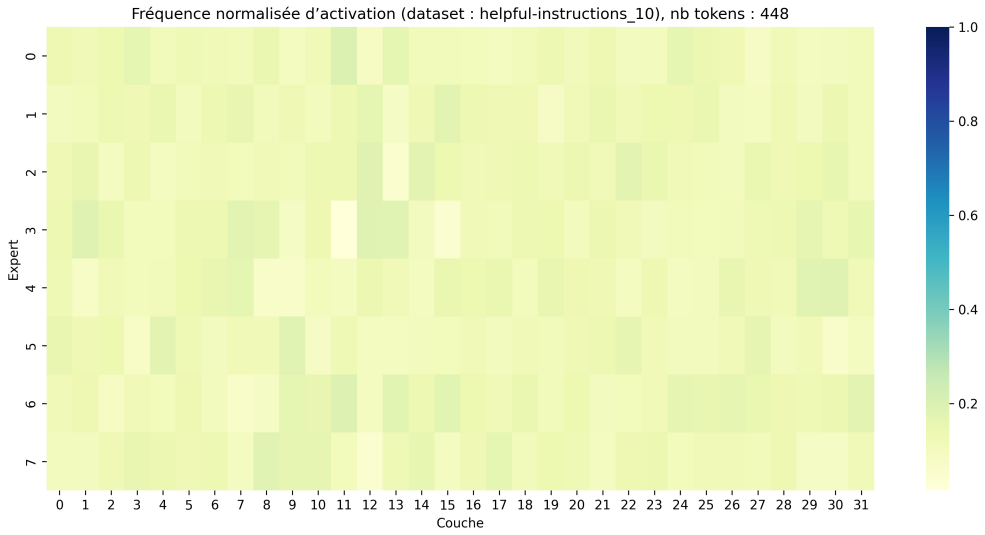



FIGURE 7 – Utilisation des experts en fonction des couches pour le jeu de données `helpful-instructions` (10 *prompts*).

Nous avons ensuite observé les trajectoires associées à un token particulier. Le token de démarrage (`<s>`), représenté figure 8, suit une trajectoire déterministe : quelle que soit l'occurrence, l'activation des experts est identique. Ce résultat est attendu puisque, en début de séquence, le contexte est vide et le modèle rencontre toujours le même état initial.

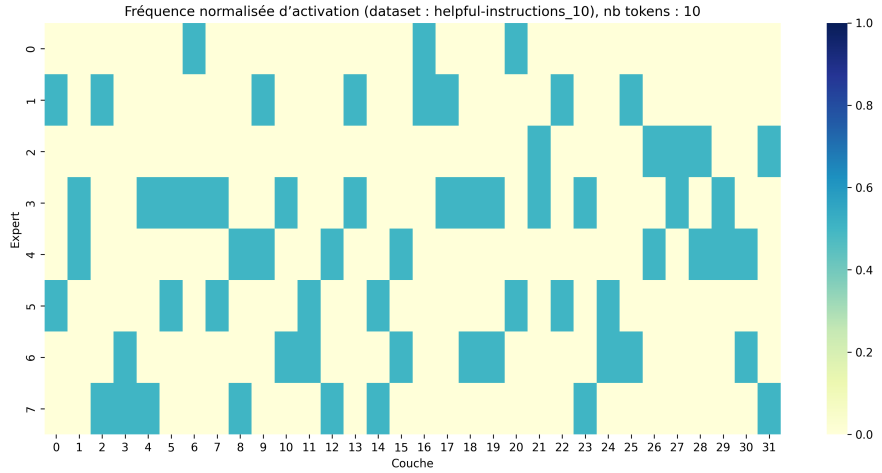



FIGURE 8 – Utilisation des experts pour le token de démarrage sur `helpful-instructions`.

À l'opposé, des tokens très dépendants du contexte, comme la virgule (id 28725), produisent des trajectoires beaucoup plus dispersées (figure 9). Il n'existe pas de chemin unique dominant, mais certains experts apparaissent néanmoins plus sollicités que d'autres. Le

token d'interrogation (id 28804) présente une variabilité similaire : son routage dépend fortement de la phrase dans laquelle il apparaît.

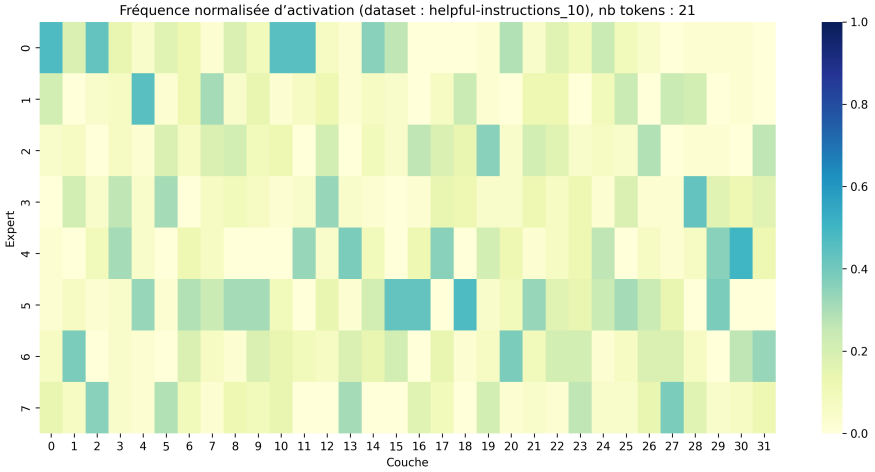



FIGURE 9 – Utilisation des experts pour le token virgule (,).

Pour affiner cette analyse, nous avons introduit une information supplémentaire : le token précédent. Par exemple, en conditionnant sur l'occurrence de la séquence `'it'` et *'?'* (ids 378 et 28804), la trajectoire devient plus nette, bien que le faible nombre d'occurrences limite la représentativité statistique.

Nous avons donc reproduit l'expérience sur un corpus plus large (10 000 *prompts*, soit environ 630 000 tokens). Les résultats pour le token `can` (id 541) sont présentés figure 10. On observe déjà quelques régularités, ce qui confirme que l'identifiant du token contient un signal partiellement prédictif. Lorsque l'on ajoute l'information du token précédent, par exemple la séquence `I can` (ids 315, 541), la trajectoire devient beaucoup plus marquée (figure 11). Certaines couches demeurent toutefois difficiles à exploiter, notamment autour des couches 16 et 17.

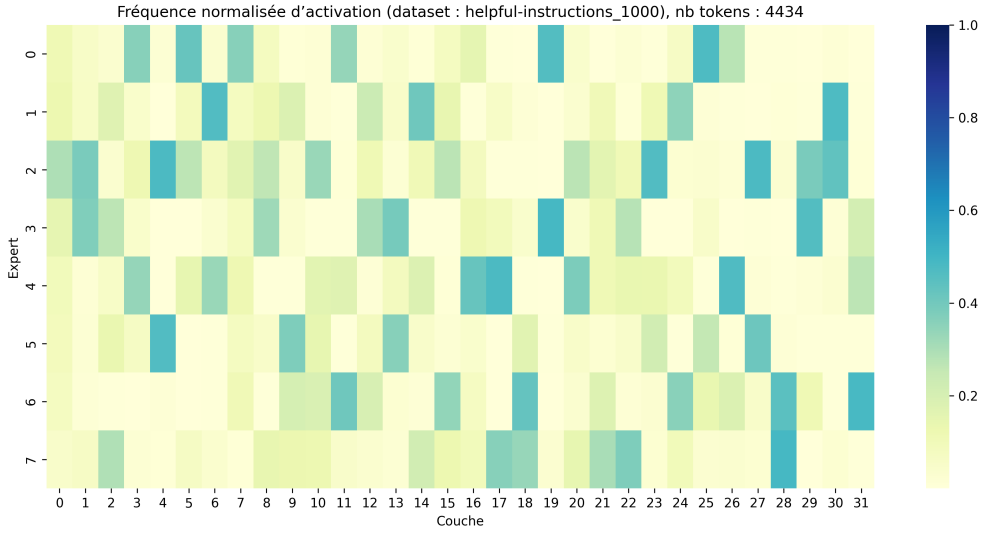



FIGURE 10 – Utilisation des experts pour le token `can` (id=541).

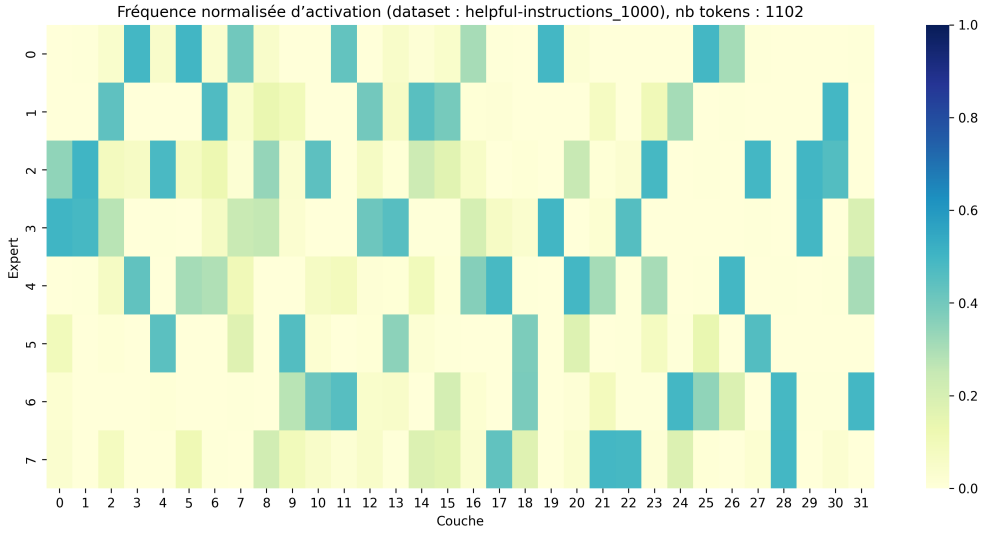



FIGURE 11 – Utilisation des experts pour le token `can` précédé du token `I`.

Ces visualisations montrent que le *token_id* seul apporte déjà une information partielle sur le routage, mais que l'ajout du *token_id* précédent permet de réduire considérablement l'ambiguïté et de faire émerger des trajectoires plus déterministes. Pour quantifier cet apport, nous avons choisi une métrique simple mais adaptée à notre problématique : le *hit rate*. Celui-ci consiste à sélectionner, pour chaque couche, les deux experts les plus probables d'après les statistiques observées et à comparer cette prédiction avec les experts effectivement utilisés lors du passage du modèle (ce qui correspond à notre métrique *top-2*.

# 5 Prédicteurs déterministes

## 5.1 Prédicteur statistique simple (profondeur 1)

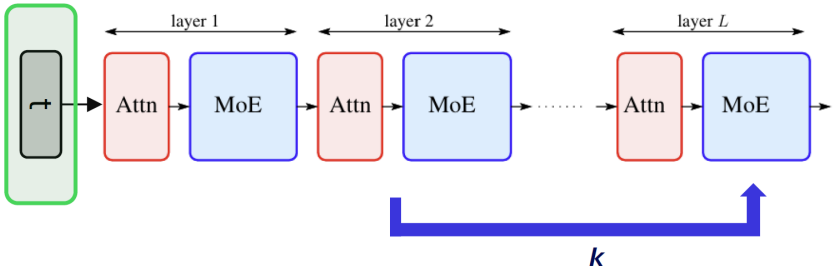



FIGURE 12 – Schéma du prédicteur statistique de profondeur 1.

La première étape de notre démarche a consisté à construire un prédicteur élémentaire, afin de tester l'hypothèse qu'il existe déjà un signal exploitable dans les seuls *token_id*. Le principe est le suivant : pour chaque *token_id*, nous parcourons les trajectoires collectées et identifions, couche par couche, la paire d'experts la plus fréquemment activée. Ce prédicteur ne conserve donc, pour chaque couche, que la paire dominante associée à ce *token_id*. Il s'agit d'une approche naïve, mais qui fournit une base de référence intéressante.

Les résultats obtenus sur les cent tokens les plus fréquents du corpus sont présentés figure 13. La précision est mesurée en *hit rate*, c'est-à-dire la proportion de couches où la paire prédite correspond effectivement aux experts sélectionnés. Nous considérons ici la métrique *top-2*, ce qui signifie qu'un succès est comptabilisé si au moins un des deux experts prédits est présent dans la paire réelle. Une précision de 100 % correspond donc à une trajectoire parfaitement prédite, avec $32 \times 2$ experts correctement anticipés.

On observe qu'en moyenne, pour les tokens fréquents, la précision atteint environ 66 % sur l'ensemble des 32 couches. Ce résultat confirme qu'il existe bien un signal statistique dans le seul identifiant du token, mais qu'il est faible pour obtenir une prédiction déterministe plus en amont. Cette première étape suggère donc d'introduire davantage d'information contextuelle, notamment en intégrant le *token_id* précédent, afin de réduire l'ambiguïté et d'améliorer la qualité des prédictions.

Nous avons également étudié la variation de la précision en fonction de la profondeur dans le réseau. La figure 14 montre que certaines couches sont plus prévisibles que d'autres : les premières couches présentent des trajectoires plus stables, tandis que les couches intermédiaires tendent à être plus variables, ce qui reflète une dépendance accrue au contexte. Mais le signal reste très faible pour être exploité.

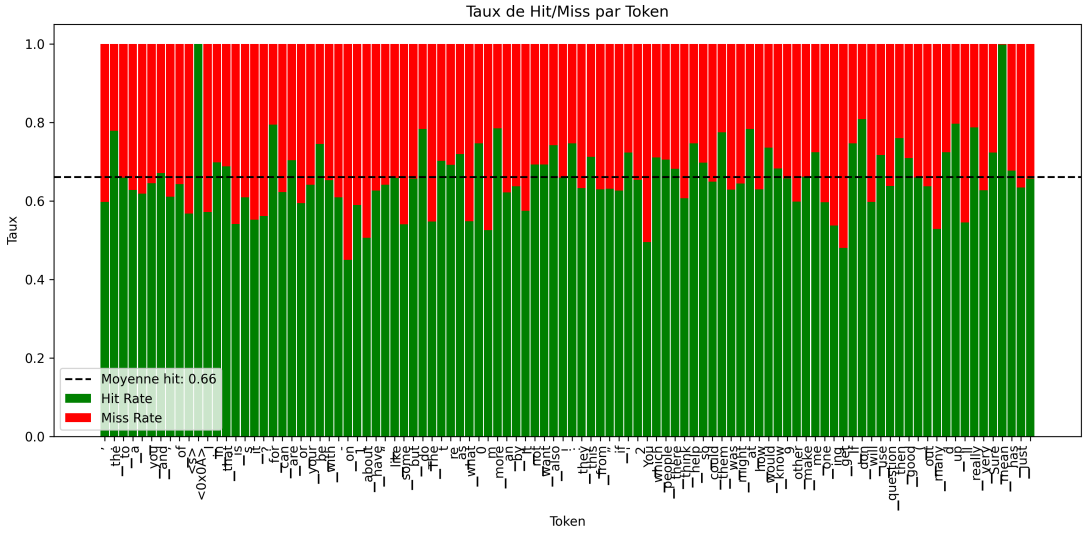



FIGURE 13 – Précision du prédicteur statistique (profondeur 1) sur les 100 tokens les plus fréquents. Chaque courbe correspond au taux de réussite couche par couche.

**Parallélisation de la construction des prédicteurs**

Pour accélérer la construction des prédicteurs statistiques, nous avons parallélisé les tâches indépendantes de calcul par *token*. Chaque prédiction pour un *token_id* donné (`single_prediction`) est autonome : on extrait sa trajectoire, on calcule les fréquences normalisées couche par couche, puis on retient les deux experts les plus probables. Cette granularité se prête naturellement à un schéma *embarrassingly parallel*.

En pratique, nous utilisons `multiprocessing.Pool` pour distribuer la fonction `single_prediction` sur plusieurs processus, avec un nombre de workers borné par `cpu_count()` et la taille du lot de tokens. Ce parallélisme permet d'exploiter simultanément tous les cœurs de calcul disponibles et de réduire drastiquement le temps de traitement des $\sim 630\,000$ tokens.

## 5.2 Extension à une profondeur supérieure

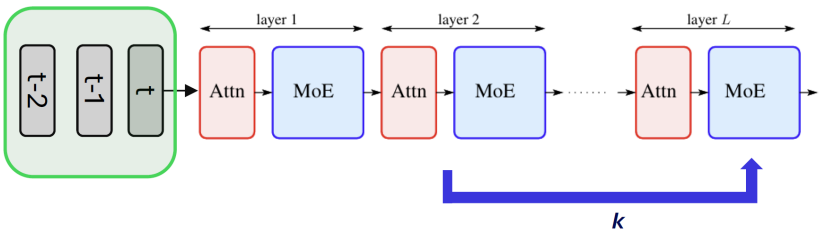



FIGURE 15 – Schéma du prédicteur statistique de profondeur $k$

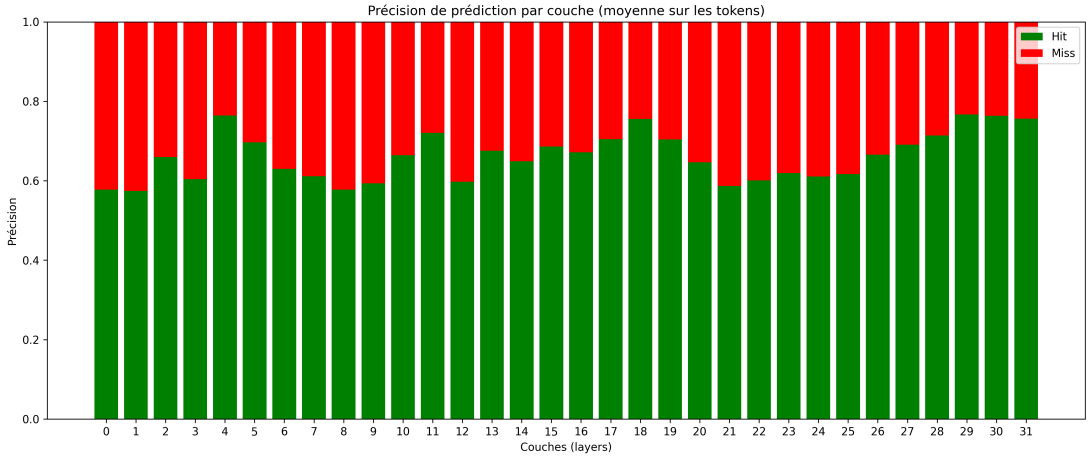



FIGURE 14 – Précision du prédicteur statistique (profondeur 1) en fonction de la couche.

Dans un second temps, nous avons cherché à enrichir le prédicteur en ajoutant du contexte. L'idée est de passer d'une entrée réduite au seul *token_id* courant à une séquence courte $(\ldots, token_{t-1}, token_t)$, c'est-à-dire à des $k$-uplets de tokens consécutifs. Pour chaque uplet observé dans les données, nous cherchons ensuite, couche par couche, la paire d'experts la plus probable.

Techniquement, la mise en œuvre est relativement simple puisque toutes les trajectoires sont déjà extraites et stockées lors des étapes précédentes. La principale difficulté vient de la taille potentiellement énorme de l'espace des $k$-uplets : avec un vocabulaire de plusieurs millions d'entrées, le nombre de combinaisons possibles croît comme $|V|^k$ (avec $V$ la taille du vocabulaire et $k$ la longueur du contexte). Afin de rendre l'approche exploitable, nous avons restreint l'analyse aux $N_p$ uplets les plus fréquents du corpus, ce qui permet déjà d'évaluer la pertinence de cette stratégie.

Nous avons testé cette approche pour une profondeur de 2 puis de 3, en conservant les 100 uplets les plus fréquents. Les résultats, présentés dans les figures suivantes, confirment une amélioration progressive de la précision :

— Pour la profondeur 2 (figure 16), la précision moyenne du prédicteur statistique augmente nettement par rapport à la profondeur 1.

— Pour la profondeur 3 (figure 17), on observe une nouvelle hausse, montrant que l'ajout de contexte permet bien de capturer davantage d'information utile.

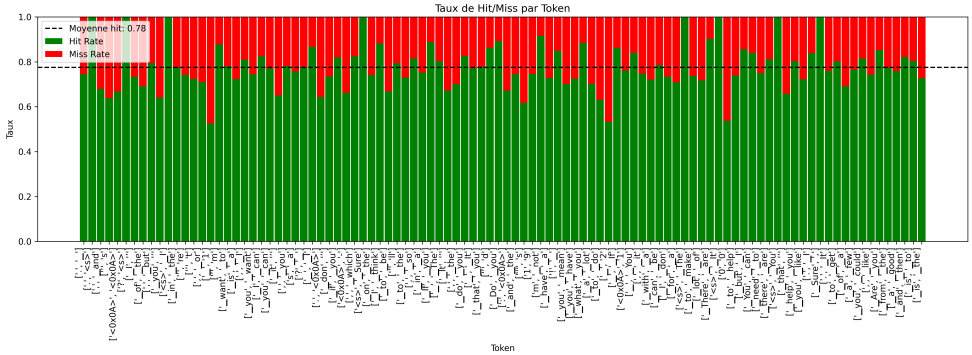



FIGURE 16 – Précision du prédicteur statistique (profondeur 2) sur les 100 uplets les plus fréquents. Pour chaque uplet, le taux de réussite (*hit rate*) est mesuré couche par couche.

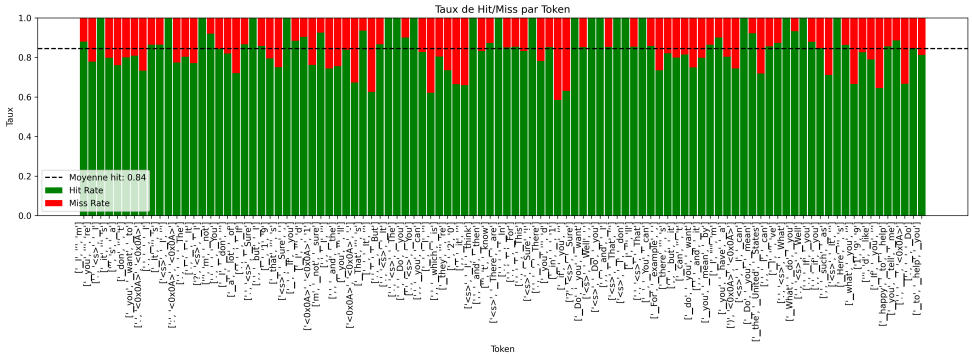



FIGURE 17 – Précision du prédicteur statistique (profondeur 3) sur les 100 uplets les plus fréquents. On observe une nouvelle amélioration par rapport à la profondeur 2.

Cette approche nous indique donc qu'il existe bien un signal exploitable dans les séquences discrètes de tokens, et que ce signal se renforce avec l'allongement du contexte. Il convient toutefois de rester prudent : les prédictions sont ici évaluées sur le même jeu de données que celui ayant servi à extraire les trajectoires, ce qui favorise mécaniquement les résultats. Nous faisons néanmoins l'hypothèse que les trajectoires sont relativement stables entre jeux de données de même nature (ici, du langage naturel en anglais), hypothèse qui reste à valider dans des travaux ultérieurs.

## 5.3 Prédicteur contextuel avec décalage

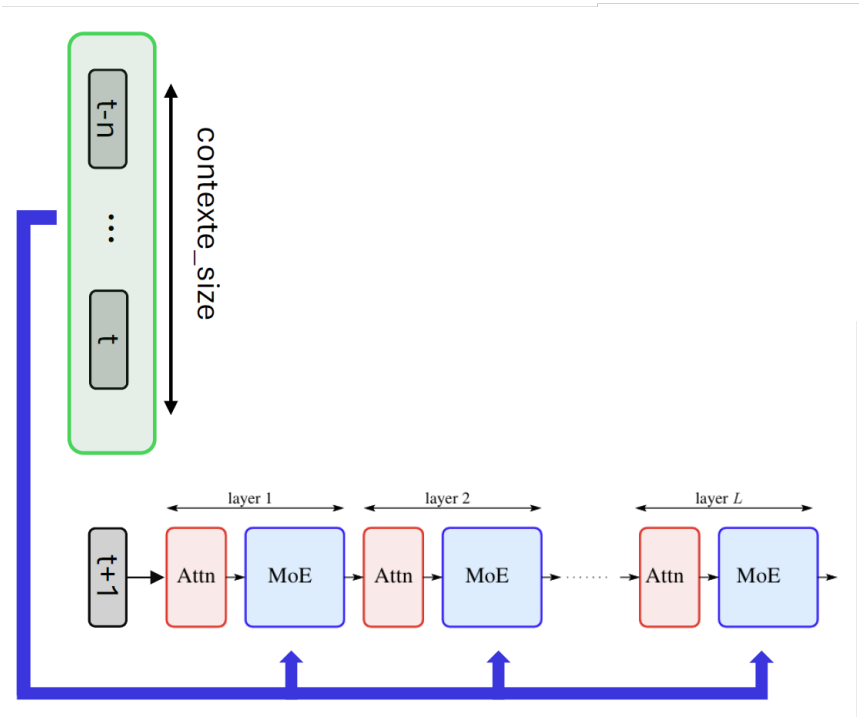



FIGURE 18 – Schéma du prédicteur contextuel avec décalage (profondeur $p$).

Après avoir testé des prédicteurs statistiques basés uniquement sur le *token_id* courant ou sur des $k$-uplets incluant ce token, nous avons exploré une variante plus ambitieuse : prédire les experts du token suivant à partir du contexte constitué uniquement des tokens précédents.

L'idée est la suivante : on prend en entrée une séquence de *token_id* de taille $p$ , $(t-p+1,\ldots,t)$, et l'on cherche à prédire la paire d'experts qui sera activée pour le token $t+1$. Cette approche présente deux motivations principales. D'une part, elle ouvre la voie à une *prédiction verticale (ou spatiale)* : lorsqu'on est à la couche $\ell$ du token $t$, on aimerait pouvoir anticiper les experts de la couche $\ell$ du token $t+1$ afin de précharger la mémoire adéquate. D'autre part, elle permet d'analyser plus finement le lien entre contexte et activation des experts, ce qui constitue un pas vers une meilleure compréhension de la dynamique interne du routage.

Dans un premier temps, nous avons simplifié l'approche : l'entrée du prédicteur est réduite à la séquence de tokens $(t-p,\ldots,t-1)$, sans inclure le token courant $t$. Pour chaque séquence observée, nous extrayons les trajectoires correspondantes et retenons, couche par couche, les experts les plus fréquents. L'évaluation se fait comme précédemment avec le *hit rate*, à la fois en fonction des couches et de la profondeur $p$.

Les résultats sont représentés en figure 19. Comme attendu, l'absence du token courant dégrade fortement la précision. Même avec une profondeur de 5, la performance reste

insuffisante pour envisager un usage pratique. On observe bien un léger gain lorsque $p$ augmente, mais celui-ci ne compense pas la perte d'information cruciale due à l'absence du token d'entrée. De plus, plus la profondeur croît, plus le nombre de séquences uniques explose, ce qui fragilise la robustesse statistique des estimations.

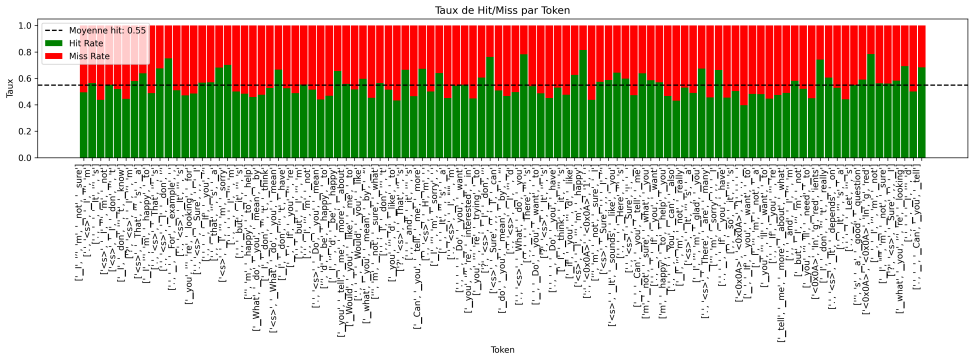



FIGURE 19 – Précision du prédicteur statistique (profondeur 5, sans inclure le token courant $t$) sur les 100 uplets les plus fréquents. Pour chaque uplet, le *hit rate* est mesuré couche par couche.

Ces limites motivent l'utilisation d'une source d'information supplémentaire : les *embeddings*. En effet, à la couche $\ell$ du token $t-1$, nous disposons d'un vecteur d'état caché de dimension 4096 qui résume non seulement le token précédent, mais aussi son contexte. Deux stratégies peuvent alors être envisagées :

1. utiliser l'embedding pour retrouver le token courant le plus probable (via un simple `argmax` sur les logits associés) et revenir ainsi au cadre des prédicteurs par *token_ id* ;
2. ou bien exploiter directement ce vecteur comme entrée du prédicteur, en l'intégrant à la séquence de tokens précédents pour enrichir le signal disponible.

## 5.4 Précision à l'échelle du vocabulaire

Jusqu'à présent, l'évaluation de nos prédicteurs reposait uniquement sur un nombre limité de cas : les $N$ tokens ou $n$-uplets les plus fréquents, avec $N$ restreint (par exemple 100) par rapport à la taille totale du vocabulaire. Cette approche exploratoire nous a permis d'identifier un signal exploitable dans les *token_ id* et de caractériser l'impact de la profondeur sur la qualité des prédictions. Toutefois, elle ne dit rien de la généralisabilité de ces résultats à l'ensemble du vocabulaire.

Pour tester cette généralisabilité, nous avons étudié la distribution des fréquences d'apparition des tokens et uplets dans un corpus plus large (10 000 *prompts*, soit environ 630 000 tokens). La figure 20 montre que la distribution des *token_ id* est fortement déséquilibrée, avec une loi proche de l'exponentielle décroissante. Une grande partie des tokens est donc très peu fréquente, et leur prise en compte risque de biaiser les mesures de précision.

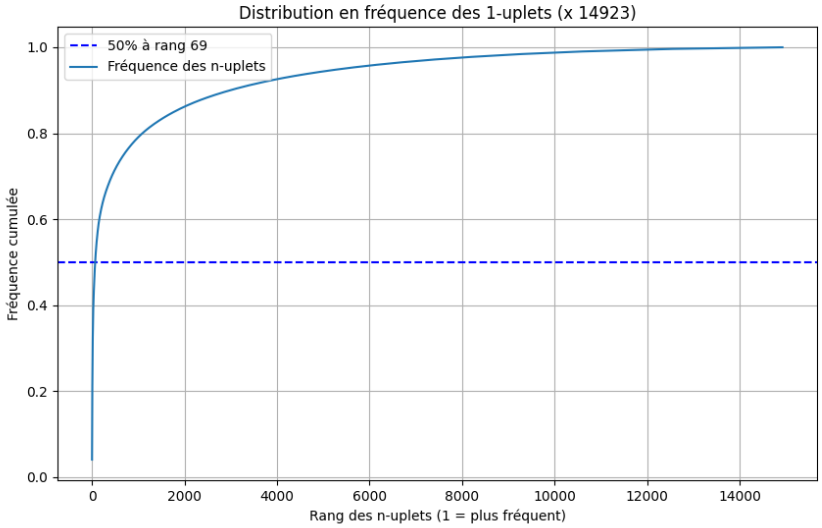



FIGURE 20 – Distribution en fréquence cumulée des *token_id* dans le dataset de 10 000 *prompts*.

Le même phénomène s'observe pour les paires (figure 21), les 3-uplets (figure 22) et les 5-uplets (figure 23), bien que la décroissance soit moins marquée que pour les tokens simples. Dans tous les cas, une minorité de séquences est très fréquente, tandis qu'une majorité est observée de manière marginale, voire une seule fois dans le corpus.

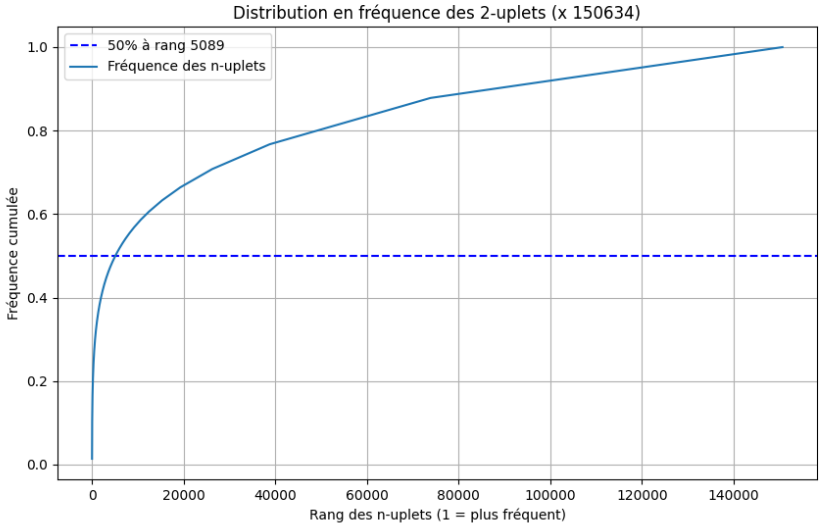



FIGURE 21 – Distribution en fréquence cumulée des paires de tokens dans le dataset de 10 000 *prompts*.

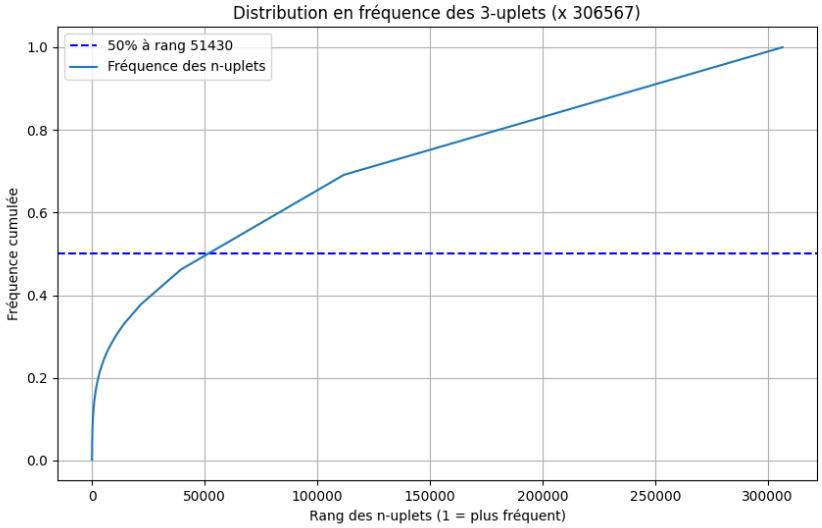



FIGURE 22 – Distribution en fréquence cumulée des 3-uplets dans le dataset de 10 000 *prompts*.

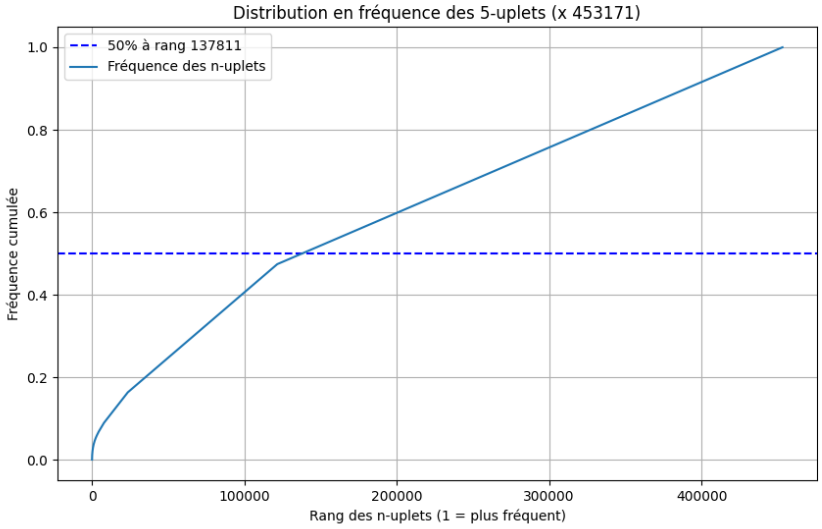



FIGURE 23 – Distribution en fréquence cumulée des 5-uplets dans le dataset de 10 000 *prompts*.

Pour illustrer l'effet de cette distribution sur la précision, nous avons évalué un prédicteur de profondeur 3 (sans décalage, le dernier token de la séquence correspondant au token d'entrée du forward). Avec $N = 100$ uplets les plus fréquents, la précision moyenne atteignait $0,84$ en *hit rate*. Lorsque nous étendons à $N = 5000$ uplets (soit environ 24% des

3-uplets distincts du corpus), la précision chute à $0,65$. Cette baisse d'environ 20 points de pourcentage reflète l'impact des séquences rares, pour lesquelles les statistiques disponibles sont beaucoup moins robustes.

Ces résultats confirment à la fois le potentiel des méthodes statistiques et leurs limites intrinsèques : la table de fréquences croît de manière exponentielle avec la profondeur, et les estimations deviennent vite bruitées lorsque l'on s'éloigne des séquences les plus fréquentes. Pour dépasser cette contrainte, nous envisageons désormais d'entraîner un **réseau de neurones léger**, capable d'apprendre les régularités de routage à partir d'un contexte réduit (par exemple deux ou trois *token_id*) et de généraliser aux cas rares. Cette transition marque le passage d'approches purement statistiques à des modèles d'apprentissage supervisé, mieux adaptés à l'échelle et à la variabilité des données.

# 6 Prédicteurs automatiques

Après avoir exploré les approches statistiques basées sur les fréquences d'activation, nous avons cherché à automatiser la prédiction des experts grâce à des réseaux de neurones. L'objectif était de dépasser les limites des tables statistiques (taille exponentielle, manque de données pour les séquences rares) et de concevoir un prédicteur capable de généraliser.

## 6.1 Préparation des données

La préparation des données constitue une étape critique. Pour chaque token du corpus, nous devons transformer les informations collectées en vecteurs utilisables par un modèle. Dans une première version, nous avons adopté la stratégie introduite à l'étape 3 (prédiction contextuelle avec décalage) : le prédicteur reçoit comme entrée une séquence de taille $N$ constituée uniquement des tokens précédents. Pour chaque token du contexte, nous concaténons son *token_ id* (un entier) et les deux experts activés à chaque couche (soit 64 entiers). L'entrée correspond donc à un vecteur aplati de dimension $(65 \times N)$.

Afin de manipuler efficacement ces vecteurs, nous les avons convertis en `torch.Tensor` et stockés au format `.pt`, ce qui permet d'alimenter directement les `DataLoader` PyTorch utilisés lors de l'entraînement.

## 6.2 Modèle MLP

Notre première tentative s'est appuyée sur un perceptron multicouche (MLP). Ce choix répondait à une logique exploratoire : le MLP est une architecture simple, rapide à implémenter, et permet de tester facilement différentes représentations en entrée.

**Principe.** Un MLP projette l'entrée aplatie dans un espace caché par des couches linéaires successives, séparées par des activations non linéaires (ReLU dans notre cas). La sortie est ensuite remodelée en logits pour chacun des 64 emplacements d'experts (32 couches $\times$ 2 experts), chaque logit correspondant à l'un des huit experts possibles. L'entraînement se fait via une perte de type *CrossEntropy*, appliquée indépendamment à chaque position.

**Architecture testée.** Nous avons testé une architecture à deux couches cachées :
- Entrée : $65 \times N$
- Projection linéaire vers 256, puis 512 neurones
- Sortie : $64 \times 8$ logits, remodelés en $(64, 8)$

**Résultats.** L'entraînement a été effectué sur GPU , avec un *batch size* de 128, optimiseur Adam et *learning rate* $10^{-4}$. Pour $N = 3$ et $N = 5$, la précision moyenne sur l'ensemble de test atteint $\sim 51,6\%$. Ce score est supérieur au hasard (25%), mais reste limité. De plus, l'ajout de contexte n'améliore pas significativement les performances, ce qui montre que la représentation choisie (tokens + experts utilisés) n'était pas optimale.

**Limites.** Cette première approche a révélé plusieurs difficultés :
- la taille des entrées augmente rapidement avec la profondeur $N$ ;
- la concaténation brute des *token_ id* et des experts ne capture pas bien la structure séquentielle ;
- le modèle reste peu expressif par rapport à la complexité du routage.

Ces constats nous ont conduit à chercher une architecture plus adaptée aux données séquentielles : le LSTM.

## 6.3 Modèle LSTM

Le LSTM (*Long Short-Term Memory*) est une variante des réseaux récurrents conçue pour traiter des séquences en tenant compte de la dépendance temporelle. Cette propriété en fait un candidat naturel pour notre tâche, où l'activation des experts dépend non seulement du token courant mais aussi du contexte constitué par les tokens précédents.

**Principe.** Le LSTM encode progressivement une séquence d'entrées en maintenant un état interne (mémoire de long terme) et un état caché (mémoire de court terme). Cela lui permet d'apprendre des régularités dans les séquences de tokens, de longueur potentiellement variable.

**Gestion des entrées.** Nous avons choisi de travailler uniquement avec les *token_ id* du contexte (sans experts activés), afin de rester dans un cadre plus générique. Chaque token est représenté par son vecteur d'embedding (dimension 4096). Cette dimension étant trop élevée pour un LSTM de petite taille, nous avons exploré deux méthodes de réduction :
- une réduction préalable par PCA (projection offline dans un espace de dimension plus faible, optimisée pour conserver la variance maximale) ;
- une première couche dense dans le réseau, qui apprend directement à compresser l'entrée en optimisant la fonction de perte.

**Expérimentations.** Nous avons testé différentes tailles de contextes ($p$ de 2 à 5), diverses dimensions projetées (par exemple 100, 200), ainsi que plusieurs tailles de couches cachées. L'entraînement a été réalisé avec la même configuration que pour le MLP (optimiseur Adam, $lr = 10^{-4}$, 20 époques, batch de 128).

**Résultats et enseignements.** Les premiers résultats montrent que le LSTM parvient mieux que le MLP à exploiter l'information séquentielle. L'usage d'un embedding réduit améliore la convergence et la stabilité de l'entraînement. Surtout, la précision augmente avec la profondeur du contexte, confirmant l'intérêt d'un modèle séquentiel.

Avec une configuration optimisée rapidement en fin de stage, nous avons obtenu les performances suivantes sur le jeu de test :

$$\text{Top-1 Accuracy} = 64.6\% \qquad \text{Top-2 Accuracy} = 33.0\%$$

Ces chiffres restent modestes, mais ils sont significativement supérieurs à la stratégie aléatoire (25 % de hit rate attendu) et montrent que le LSTM est capable de capter une partie de la structure sous-jacente au routage.

**Transition.** En résumé, le MLP a servi de banc d'essai pour tester rapidement des représentations, mais il atteint vite ses limites. Le LSTM, en revanche, est plus adapté à la nature séquentielle des données et ouvre la voie à des prédicteurs capables de généraliser au-delà des simples statistiques de fréquences. Cette partie a été réalisée à la fin du stage dans un temps limité mais mérite d'être explorée plus en détails.

# 7 Conclusion et Perspectives

## Bilan

Ce travail a exploré une trajectoire progressive, des statistiques descriptives au réseau neuronal, pour *prédire* le routage d'experts dans un LLM de type MoE et *précharger* les bons experts afin de masquer les E/S sous le calcul. Trois constats principaux émergent :

1. **Le routage local est fortement structuré.** La réutilisation du *gating network* (prédiction spatiale à $n$=1) atteint des précisions élevées (top-1 ∼96 %, top-2 ∼90 %), validant l'idée d'un préchargement « à une couche d'avance » peu coûteux et efficace.
2. **Le *token_id* contient un signal exploitable, renforcé par le contexte.** Les prédicteurs statistiques de profondeur 1 (sur tokens fréquents) atteignent ∼66 % en top-2, et montent jusqu'à ∼84 % pour des 3-uplets très fréquents (au prix d'une moindre généralisabilité lorsque l'on élargit $N$). En décalage (sans le token courant), la précision chute, ce qui souligne le rôle central de l'information la plus récente.
3. **Les modèles séquentiels légers généralisent mieux que le MLP.** Le MLP (entrée aplatie) dépasse l'aléatoire mais plafonne (∼51,6 % top-2), alors qu'un LSTM simple, avec réduction d'embeddings, semble mieux capter des régularités temporelles utiles tout en restant compact.

TABLE 1 – Récapitulatif des approches, entrées et précisions obtenues.

| **Méthode** | **Entrée** | **Top-1** | **Top-2** |
|---|---|---|---|
| Aléatoire | — | 12.5 % | 25 % |
| Gating (spatial $n$=1) | $h_\ell$ appliqué par $g_{\ell+1}$ | ∼96 % | ∼90 % |
| Stat. prof. 1 (tokens fréquents) | token_id | — | ∼66 % |
| Stat. prof. 3 (sans décalage) | 3-uplets | — | ∼84 % (100 uplets) → 65 % (5k) |
| Stat. décalé ($p$=5) | tokens $t-p$:$t-1$ | — | 55 % |
| MLP (entrée aplatie) | concat. tokens & experts | — | ∼51,6 % |
| **LSTM (+ réduction)** | tokens (embeddings réduits) | **64,6 %** | **33%** |

## Perspectives

Plusieurs pistes concrètes se dégagent pour transformer ces résultats en gains d'inférence mesurables à l'échelle système :

— **Intégration système du préchargement.** Implémenter une politique de *prefetch* par couche (fenêtre, budget mémoire, seuils de confiance) combinant prédictions *temporelles* (grossières) et *spatiales* (affinage tardif).

— **Combinaison de signaux.** Fusionner statistiques (priors par token/uplet), logits du routeur et embeddings réduits (PCA ou couche apprise) dans un petit classifieur séquentiel (LSTM/GRU ou Transformer léger) calibré pour le *top-2*.

- **Réduction dimensionnelle pilotée par la tâche.** Évaluer systématiquement le compromis variance expliquée (PCA) $\leftrightarrow$ précision de prédiction, ou apprendre une compression dédiée (couche linéaire gelée/apprise) pour minimiser la perte utile.
- **Généralisation et robustesse.** Valider *cross-domaine* (autres corpus, autres MoE), étudier l'impact des *router noise/jitter*, et quantifier l'apport d'un *calibrage* des probabilités (température, focal loss) pour des politiques *cost-aware*.
- **Gains bout-en-bout.** Mesurer les économies de latence mur et d'E/S selon la VRAM disponible (cas « pleinement chargé » vs « partiellement chargé »), ainsi que les compromis mémoire/temps via quantification légère d'experts en *standby*.

**En synthèse**, la prédiction d'experts est faisable, utile pour le préchargement et déjà performante à courte portée (gating $n$=1). Les approches statistiques révèlent où se trouve l'information, et un LSTM léger, couplé à des embeddings réduits, offre une voie praticable pour anticiper *à l'avance* sans exploser le coût. La prochaine étape consiste à *fermer la boucle* avec une implémentation système de préfetch/caching et une évaluation bout-en-bout des gains d'inférence.

# Remerciements

Je tiens à exprimer ma profonde gratitude à **Loris Marchal** pour son accompagnement attentif tout au long du stage, sa grande disponibilité, ainsi que pour l'organisation du *Workshop on Scheduling in Montreal*[1] et l'opportunité qu'il m'a offerte de présenter mes travaux.

Je remercie également **Pablo Piantanida** pour son soutien continu, son implication dans le suivi administratif et pour son accueil chaleureux au sein de l'ILLS.

1. `https://perso.ens-lyon.fr/loris.marchal/scheduling-in-montreal/`

# 8 Bibliographie

## Résumé des références


[1] Antoine. "POS-specialization in MoE Routing". In : *arXiv preprint* (2025). En préparation.

[2] William Fedus, Barret Zoph et Noam Shazeer. "Switch Transformers : Scaling to trillion parameter models with simple and efficient sparsity". In : *arXiv preprint arXiv :2101.03961* (2021).

[3] Zhiqi He et al. "ExpertFlow : Efficient Expert Offloading in MoE Models". In : *arXiv preprint arXiv :2407.14465* (2024).

[4] Robert A Jacobs et al. "Adaptive mixtures of local experts". In : *Neural Computation* 3.1 (1991), p. 79-87.

[5] Simon Lo et al. "A Closer Look at the Behavior of Experts in MoE Language Models". In : *arXiv preprint arXiv :2402.11130* (2024).

[6] Noam Shazeer et al. "Outrageously Large Neural Networks : The sparsely-gated Mixture-of-Experts layer". In : *arXiv preprint arXiv :1701.06538* (2017).

[7] Zhenyu Tang et al. "Hobbit : Efficient Quantization for MoE Models". In : *arXiv preprint arXiv :2406.09491* (2024).

[8] Eli Trajtenberg et al. "Trajectories of Experts in Mixture-of-Experts". In : *arXiv preprint arXiv :2409.20085* (2024).

[9] Itay Yona et al. "Stealing User Prompts from Mixture of Experts". In : *arXiv preprint arXiv :2410.22884* (2024).

[10] Tianyang Zhong et al. "AdapMoE : Adaptive Sensitivity Mixture-of-Experts for Efficient Inference". In : *arXiv preprint arXiv :2402.10954* (2024).